

Security Review

Cantina Managed

COINBASE: BASE DYNAMIC-UPGRADES

Cantina Managed review by:

Sujith Somraaj, Lead Security Researcher

Cryptara, Security Researcher



September 19, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	High Risk	4
3.1.1	Untrusted <code>RollupConfig</code> can drive noncanonical proof execution under a canonical public commitment	4
3.2	Medium Risk	6
3.2.1	Unordered runtime schedule updates can roll back consensus rules and change them during block execution	6
3.2.2	Live monitor advances L1 baseline before apply, permanently suppressing failed auto-reapply (including incident delays)	9
3.2.3	Runtime schedule application accepts past activation timestamps and clears already-active upgrades	10
3.2.4	Default finalized polling can learn emergency delays only after the superseded activation	12
3.2.5	Runtime schedule updates leave Reth's P2P fork filter stale, partitioning running and restarted nodes	13
3.2.6	EIP-8130 activation can disable ZK challenges before compatible verifier rotation	14
3.3	Low Risk	16
3.3.1	Standalone CL and EL can run with different upgrade schedules without detection	16
3.3.2	Live monitor misses schedule shrinkage following an L1 reorganization	16
3.3.3	Unknown scheduled upgrades are ignored based on an unenforced compatibility assumption	17
3.3.4	Upgrade schedule reads are not bound to the configured L1 chain	18
3.3.5	Upgrade-signal reads lack consistent resource limits	18
3.3.6	The <code>UncommittedZenithUpgrade</code> rejection uses the pin timestamp rather than the claim timestamp	19
3.3.7	Empty <code>getSchedule</code> never auto-applies and differs from all-zero <code>Never / clear_chain</code> encodings	19
3.3.8	Optional boot preimage reads silently default on all oracle errors	21
3.3.9	<code>AggregateVerifier</code> permanently pins a still-mutable future schedule	21
3.4	Informational	23
3.4.1	Contract-valid dependency holes cause inconsistent CL and EL fork activation	23
3.4.2	Development builds bypass protocol-version compatibility checks	25
3.4.3	Schedule commitment does not bind numeric positions to semantic upgrade identities	25
3.4.4	Runtime-admin can activate <code>Beryl</code> without the required activation admin	26
3.4.5	The <code>minimumProtocolVersion</code> gate compares packed versions as raw <code>U256</code> , inverting prerelease ordering	28
3.4.6	<code>ScheduleId::pin</code> doc cites <code>scheduleId()</code> while implementing activated-prefix semantics	29
3.4.7	<code>Clone</code> does not guarantee transactional sink isolation	29
3.4.8	Runtime-added upgrades are omitted from fork enumeration	30



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

Base is a secure, low-cost, builder-friendly Ethereum L2 built to bring the next billion users onchain.

From Aug 10th to Aug 18th the Cantina team conducted a review of [base](#) on commit hash [e1a8cf1d](#). The team identified a total of **24** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	6	6	0
Low Risk	9	5	4
Gas Optimizations	0	0	0
Informational	8	3	5
Total	24	15	9

2.1 Scope

The security review had the following components in scope for [base](#) on commit hash [e1a8cf1d](#):

```
crates
├── consensus/service/src/actors/upgrade_signal.rs
├── execution/cli/src/upgrade_signal.rs
├── proof/proof/src/boot.rs/schedule_id.rs
├── utilities/upgrade-signal/src
│   ├── contract.rs
│   ├── error.rs
│   ├── ids.rs
│   ├── lib.rs
│   ├── metrics.rs
│   ├── state.rs
│   ├── bin/contract_upgrade_ids.rs
│   ├── config
│   │   ├── args.rs
│   │   ├── mod.rs
│   │   ├── schedule.rs
│   │   └── types.rs
│   └── runtime
│       ├── mod.rs
│       ├── refresher.rs
│       ├── sink.rs
│       └── summary.rs
```

3 Findings

3.1 High Risk

3.1.1 Untrusted `RollupConfig` can drive noncanonical proof execution under a canonical public commitment

Severity: High Risk.

Context: [boot.rs#L264-L289](#).

Description: The proof host supplies a serialized `RollupConfig` through the local pre-image oracle. Current implementation accepts this complete configuration after validating only that its `l2_chain_id` matches the boot chain ID.

The complete configuration is subsequently used for L2 derivation. However, the public `rollupConfigHash` is calculated from `crates/proof/primitives/src/per_chain_config.rs:127`, which omits derivation-critical fields such as:

- `batch_inbox_address`
- `seq_window_size`
- `max_sequencer_drift`
- Channel timeouts.
- Blob configuration.
- `protocol_versions_address`
- L1 chain ID and fee configuration.

Additionally, `crates/proof/primitives/src/per_chain_config.rs:205` normalizes `block_time` and the genesis L2 block number before hashing. `scheduleId` only commits upgrade activation timestamps and therefore does not cover these static parameters.

Consequently, it is possible to construct configurations `C` and `C'` such that:

```
rollupConfigHash(C) == rollupConfigHash(C')
scheduleId(C)       == scheduleId(C')
Derive(C, L1Data)   != Derive(C', L1Data)
```

A malicious or compromised proof host can copy the canonical configuration, modify an omitted field, and provide it to the guest. The zkVM then legitimately proves execution under the modified rules, while its public output carries the expected canonical commitment. This is not a failure of SNARK soundness; the proven statement does not sufficiently bind the executed configuration.

The Nitro path has the same trust mismatch: it executes using `boot_info.rollup_config`, but obtains the signed `crates/proof/tee/nitro-enclave/src/server.rs:149`.

A compromised authorized prover could therefore potentially produce an accepted proof or enclave signature for a noncanonical output root, affecting disputes and any downstream state-root acceptance or withdrawal mechanism.

This affects versions that load oracle-supplied configurations for built-in chains. Base v1.2.0 does not contain that behavior for known Base chain IDs.

Proof of Concept: Place the following test in `etc/systems/tests/proof_rollup_config_commitment.rs`:

```
/// The secure invariant is that changing a derivation-critical field must either
↪ make
```



```

/// ['BootInfo::load'] fail or change the public rollup-config commitment. This
↳ 'PoC' asserts the
/// current inverse behavior so it passes only while the vulnerability is
↳ reproducible.
#[tokio::test]
async fn mutated_config_passes_boot_under_the_same_public_commitment() ->
↳ Result<()> {
    let canonical_config = BaseChainConfig::MAINNET.rollup_config();
    let l1_head = B256::repeat_byte(0x11);
    let agreed_output_root = B256::repeat_byte(0x22);
    let claimed_output_root = B256::repeat_byte(0x33);

    let canonical_oracle = BootOracle::new(
        &canonical_config,
        l1_head,
        agreed_output_root,
        claimed_output_root,
        CLAIM_BLOCK_NUMBER,
    )?;
    let canonical_boot = BootInfo::load(&canonical_oracle)
        .await
        .wrap_err("canonical node config should pass proof boot")?;

    let mut mutated_config = canonical_config;
    mutated_config.seq_window_size = mutated_config
        .seq_window_size
        .checked_add(1)
        .ok_or_eyre("Base Mainnet sequence window overflowed")?;
    let mutated_oracle = BootOracle::new(
        &mutated_config,
        l1_head,
        agreed_output_root,
        claimed_output_root,
        CLAIM_BLOCK_NUMBER,
    )?;
    let mutated_boot = BootInfo::load(&mutated_oracle)
        .await
        .wrap_err("PoC no longer reproduces: proof boot rejected the mutated
↳ config")?;

    assert_eq!(
        mutated_boot.rollup_config.seq_window_size, mutated_config.seq_window_size,
        "proof boot must execute the attacker-supplied derivation parameter for this
↳ PoC",
    );
    assert_eq!(
        PublicRollupConfigCommitment::hash(&canonical_boot.rollup_config)?,
        PublicRollupConfigCommitment::hash(&mutated_boot.rollup_config)?,
        "PoC no longer reproduces: the mutation changed the public
↳ rollupConfigHash",
    );

    Ok(())
}

```

The test:

1. Loads the compiled Base Mainnet configuration.
2. Changes only `seq_window_size`.
3. Supplies the modified configuration under `L2_ROLLUP_CONFIG_KEY`.
4. Confirms `BootInfo::load` accepts and retains the mutation.
5. Confirms both configurations produce the same public `rollupConfigHash`.



Run it locally with:

```
cargo test -p base-system-tests \
  --no-default-features \
  --test proof_rollup_config_commitment \
  -- --nocapture
```

Result:

```
test mutated_config_passes_boot_under_the_same_public_commitment ... ok
test result: ok. 1 passed; 0 failed
```

Recommendation: For built-in chains, construct the full `RollupConfig` from trusted compiled chain data. Accept only dynamic upgrade timestamps from the external source, and bind those separately through the validated `scheduleId`.

Coinbase: Fixed for supported chains by [PR-4387](#). Unknown/dev chains retain a trust-based fallback.

Cantina Managed: Verified fix.

3.2 Medium Risk

3.2.1 Unordered runtime schedule updates can roll back consensus rules and change them during block execution

Severity: Medium Risk.

Context: [sink.rs#L41-L54](#).

Description: The runtime upgrade registry is a process-global map keyed only by L2 chain ID. Although access is protected by an `RwLock`, an installed schedule does not include its L1 block number, block hash, or schedule ID. Consequently, the lock makes individual writes atomic but does not prevent an older request from overwriting newer state.

`crates/common/genesis/src/chain/upgrade.rs:429` unconditionally replaces the current overrides:

```
pub fn replace_overrides(chain_id: u64, overrides: UpgradeActivationOverrides) {
    Self::write_registry().insert(chain_id, overrides);
}
```

Similarly, `crates/utilities/upgrade-signal/src/runtime/sink.rs:41` commits every completed refresh without comparing its source against the currently installed schedule.

This permits the following ordering:

1. Refresh A begins reading the schedule at L1 block 100.
2. Refresh B begins reading the schedule at L1 block 101.
3. B completes first and installs the newer block-101 schedule.
4. A completes later and unconditionally replaces it with the stale block-100 schedule.

The reader pins contract calls to an L1 block hash, but the hash is discarded after the read. The resulting `crates/utilities/upgrade-signal/src/state.rs:13` retains only the block number, and even that number is used only for reporting rather than commit ordering. The contract's canonical `scheduleId` is not read.

Additionally, fork checks do not capture an immutable schedule. `crates/execution/chainspec/src/spec.rs:292` consults the process-global registry on every call. A refresh can therefore change consensus rules during an operation that makes multiple fork checks.

A concrete example exists in block execution:



- Before executing a transaction, the executor checks whether Jovian is active and calculates the transaction's DA footprint at `crates/common/evm/src/executor/block_executor.rs:230`.
- After execution, it checks Jovian again before adding that footprint to the block total at `crates/common/evm/src/executor/block_executor.rs:292`.

If a refresh occurs between these checks, the transaction can be evaluated under two different rule sets. For example, Jovian can be inactive during the first check but active during the second, causing the transaction to bypass the Jovian DA-footprint validation while committing a zero footprint.

Because refresh timing differs across nodes, one node may build or accept a block that another node rejects under a consistent schedule. This can cause consensus divergence or a chain split.

Proof of Concept: The following deterministic test models a newer L1 read completing first and a slower, older read completing afterward. It then demonstrates that live fork checks for the same block timestamp return different results within one simulated operation.

Add the following test before the final closing brace of the existing `#[cfg(test)]` mod tests in `crates/utilities/upgrade-signal/src/runtime/sink.rs`:

```
#[test]
fn stale_l1_schedule_rolls_back_rules_during_one_operation() {
    const CHAIN_ID: u64 = 9_000_010;
    const BLOCK_TIMESTAMP: u64 = 150;

    RuntimeUpgradeRegistry::clear_chain(CHAIN_ID);
    let config = base_common_genesis::RollupConfig {
        l2_chain_id: CHAIN_ID.into(),
        ..base_common_genesis::RollupConfig::default()
    };

    // The newer L1 read finishes first. Jovian is inactive for this block.
    let mut newer = schedule(&[(BaseUpgrade::Jovian, 200)]);
    newer.signals[0].l1_block_number = 101;

    let newer_summary =
        UpgradeSignalRuntimeApplier::apply_schedule(CHAIN_ID, &newer);

    assert_eq!(newer_summary.l1_block_number, Some(101));

    // Models the executor's first fork check.
    let rules_before_transaction =
        config.is_jovian_active(BLOCK_TIMESTAMP);

    assert!(!rules_before_transaction);

    // The older, slower L1 read finishes later. It is accepted even though
    // block 101 was already committed.
    let mut stale = schedule(&[(BaseUpgrade::Jovian, 100)]);
    stale.signals[0].l1_block_number = 100;

    let stale_summary =
        UpgradeSignalRuntimeApplier::apply_schedule(CHAIN_ID, &stale);

    assert_eq!(stale_summary.l1_block_number, Some(100));

    // Models the executor's second fork check for the same block.
    let rules_after_transaction =
        config.is_jovian_active(BLOCK_TIMESTAMP);

    assert!(rules_after_transaction);

    // The same operation observed two different consensus rule sets.
    assert_ne!(rules_before_transaction, rules_after_transaction);
}
```



```
// The stale block-100 schedule replaced block 101.
assert_eq!(
    RuntimeUpgradeRegistry::activation(
        CHAIN_ID,
        BaseUpgrade::Jovian
    ),
    Some(UpgradeActivation::Timestamp(100))
);

RuntimeUpgradeRegistry::clear_chain(CHAIN_ID);
}
```

Run:

```
cargo test -p base-upgrade-signal \
    stale_l1_schedule_rolls_back_rules_during_one_operation \
    -- --nocapture
```

Expected result:

```
running 1 test
test runtime::sink::tests::stale_l1_schedule_rolls_back_rules_during_one_operation
↳ ... ok

test result: ok. 1 passed; 0 failed
```

The passing test proves that:

1. The block-101 schedule is installed successfully.
2. The block-100 schedule is subsequently accepted and replaces it.
3. A fork check for timestamp 150 changes from pre-Jovian to Jovian without changing the local configuration object or block timestamp.

Recommendation: Represent each installed schedule as an immutable, versioned epoch containing at least:

- L1 chain ID and contract address.
- L1 block number and block hash.
- Contract `scheduleId`.
- Complete activation overrides.

Route automatic execution polling, consensus polling, and manual admin refreshes through one node-scoped writer. Reject commits from lower L1 block numbers. Treat the same block number with a different block hash or schedule ID as a conflict requiring explicit reorg handling.

For safe or latest reads, implement a canonicity-aware reorg procedure instead of accepting arbitrary rollback. Production operation should preferably consume finalized schedules.

Finally, capture an immutable schedule snapshot at the beginning of every payload build, block execution, block validation, and derivation operation. Pass that snapshot through the entire operation so that no consensus-critical code consults mutable global state mid-operation.

Coinbase: Fixed in [PR-4396](#).

Cantina Managed: Verified fix. The lower-block overwrite is fixed. Equal-height reorg conflicts and per-operation schedule snapshots remain unaddressed, but finalized polling and the contract's notice/freeze constraints substantially limit their reachability and is an accepted-risk.



3.2.2 Live monitor advances L1 baseline before apply, permanently suppressing failed auto-reapply (including incident delays)

Severity: Medium Risk.

Context: `upgrade_signal.rs#L103-L116`, `upgrade_signal.rs#L165-L185`, `schedule.rs#L71-L88`, `refresher.rs#L35-L57`, `state.rs#L70-L151`.

Description: RuntimeAdmin live auto-apply is split across two steps that do not share failure semantics.

`UpgradeSignalMonitor::poll` reads L1, writes the observed values into `states`, and returns `Some(schedule)` when any signal is `Initialized` or `Changed`. The EL observer and the CL actor then call `UpgradeSignalRefresher::apply`. `apply` validates the global `minimumProtocolVersion` attached to every positive-timestamp signal and only then mutates `RuntimeUpgradeRegistry`.

If validation fails, the registry is unchanged but the monitor baseline has already moved. The next poll compares against that poisoned baseline, classifies the same L1 values as `Unchanged`, and returns `None`. Auto-apply never retries. A later binary upgrade that would make the same schedule valid still does not re-trigger the live path, because the baseline already holds those contract values.

The refresher documents that failed reads never advance the monitor baseline. That is true for read failures. It is false for apply failures.

The version gate makes this operationally severe. Solidity `minimumProtocolVersion` is informational and is not part of `scheduleId`; the owner can bump it independently of timestamps. Rust treats it as a hard gate on the entire schedule. `validate_signal_supported_protocol_version` skips the gate only when `activation_timestamp == 0` (clears). An incident `delayTimestamp` must keep a positive timestamp, so a delay of an already-supported fork is rejected on an old binary whenever the global minimum has been raised.

Honest composition:

1. Owner raises `minimumProtocolVersion` because a new binary exists.
2. Incident responder later delays an already-supported fork.
3. An un-upgraded RuntimeAdmin node reads both changes, fails `apply`, poisons its baseline, and keeps the old activation.
4. Upgraded nodes apply the delay. At the original timestamp the old node forks.

The node does not halt. It only warns and continues producing or validating with stale activations. Metrics also record the observed (rejected) signal before apply, so dashboards can show the delayed timestamp as if it had been adopted.

Proof of concept: Integration test in `crates/utilities/upgrade-signal/src/state.rs` (`monitor_does_not_reoffer_schedule_after_failed_apply`). It pins explicit protocol versions (so a workspace `0.0.0` / `U256::MAX` binary cannot mask the gate), commits the rejected schedule into the monitor baseline, shows `apply` fails without mutating the registry, shows the next observation is `Unchanged` / `not requires_apply`, and shows that even after the binary gains support the live monitor still will not re-offer the same L1 values.

```
#[test]
fn monitor_does_not_reoffer_schedule_after_failed_apply() {
    let chain_id = 9_100_020;
    RuntimeUpgradeRegistry::clear_chain(chain_id);

    let supported = UpgradeSignalDefaults::packed_protocol_version(1, 1, 0);
    let unsupported = UpgradeSignalDefaults::packed_protocol_version(1, 2, 0);
    let mut config = UpgradeSignalConfig::new(Address::ZERO);
    config.node_protocol_version = supported;
```



```

let mut monitor =
  ↳ UpgradeSignalMonitor::new(UpgradeSignalMetricLayer::Consensus);
let rejected = UpgradeSignalSchedule::new(vec![UpgradeSignal {
  upgrade_id: BaseUpgrade::Azul,
  activation_timestamp: 42,
  protocol_version: unsupported,
  l1_block_number: 1,
}]);

assert_eq!(
  monitor.update_schedule(rejected.clone()),
  vec![UpgradeSignalStateUpdate::Initialized]
);

let refresher = UpgradeSignalRefresher::new(
  config.clone(),
  RootProvider::new_http("http://127.0.0.1:1".parse().unwrap()),
  chain_id,
  UpgradeSignalMetricLayer::Consensus,
);
assert!(refresher.apply(&rejected).is_err());
assert_eq!(RuntimeUpgradeRegistry::activation(chain_id, BaseUpgrade::Azul),
  ↳ None);

let second = monitor.update_schedule(rejected.clone());
assert_eq!(second, vec![UpgradeSignalStateUpdate::Unchanged]);
assert!(!second[0].requires_apply());

config.node_protocol_version = unsupported;
let upgraded = UpgradeSignalRefresher::new(
  config,
  RootProvider::new_http("http://127.0.0.1:1".parse().unwrap()),
  chain_id,
  UpgradeSignalMetricLayer::Consensus,
);
assert!(upgraded.apply(&rejected).is_ok());
assert!(!monitor.update_schedule(rejected)[0].requires_apply());

RuntimeUpgradeRegistry::clear_chain(chain_id);
}

```

Run: `cargo test -p base-upgrade-signal --lib monitor_does_not_reoffer_schedule_after_failed_apply` .

Recommendation: Do not commit monitor state until `apply` succeeds. On apply failure, leave the previous baseline so the next poll re-detects `Changed` . Fail closed in RuntimeAdmin when a live apply fails rather than warning and continuing. Narrow the version gate so delays and clears of forks this binary already implements are still applied. Instrument apply failures separately from observed L1 signals.

Coinbase: Fixed in [PR-4513](#).

Cantina Managed: Verified fix. The baseline advances only after successful application.

3.2.3 Runtime schedule application accepts past activation timestamps and clears already-active upgrades

Severity: Medium Risk.

Context: [upgrade_signal.rs#L103-L116](#), [upgrade_signal.rs#L165-L185](#), [refresher.rs#L39-L57](#), [sink.rs#L41-L104](#).

Description:



Summary: RuntimeAdmin applies L1 upgrade schedules with no check against the node's current L2 head. An honest pre-activation clear or delay that is observed after L2 has already crossed the old activation timestamp rewrites fork interpretation for blocks the node has already produced, because runtime overrides take precedence for all timestamps.

Details: The only validation applied to a live schedule before it mutates a running node's fork configuration is the protocol-version check in `UpgradeSignalRefresher::apply . apply_schedule_to_sink` then writes every signal unconditionally through `UpgradeActivation::from_timestamp`.

There is no check that an activation timestamp is in the future relative to the node's current head, that an already-active upgrade is not being cleared, or that the new value for an already-activated upgrade equals the value the node has been running with.

In RuntimeAdmin mode this happens automatically: the CL actor and the EL node-started task call `refresher.apply` as soon as the monitor reports a change. `RuntimeRegistrySink::finalize` publishes via `RuntimeUpgradeRegistry::replace_overrides`, and `RollupConfig::upgrade_activation` gives that override precedence for all timestamps, past and future alike.

Because fork predicates are pure functions of `(timestamp, activation)` with no memory of what was used when a block was first executed, changing the activation changes the node's interpretation of history. Setting a past Azul activation retro-activates Azul for historical blocks; clearing Beryl with `0` retro-deactivates a live fork the same way.

This does not require a malicious privileged role. Solidity permits past timestamps during registration and permits clearing until L1 time reaches activation. A clear or delay submitted before activation can be observed through finalized polling after L2 has already crossed the old activation, so an honest incident response plus default finalized lag is enough to rewrite fork interpretation for already-produced blocks.

Impact and likelihood: Impact is high because already-executed blocks can be reinterpreted under different fork rules (re-execution, ForkId / peer matching, and shared EL+CL sequencer paths). Likelihood is medium because the trigger is an honest incident clear/delay combined with default finalized observation lag near an activation boundary, not a malicious admin action and not every upgrade event.

Proof of concept: Integration test in `crates/utilities/upgrade-signal/src/runtime/sink.rs` (`runtime_apply_allows_retroactive_fork_activation`). It builds a `RollupConfig` keyed to a chain id, shows Azul inactive and Beryl active at a historical head timestamp, applies a runtime schedule that installs a past Azul activation and clears Beryl, then shows the same historical timestamp is reinterpreted.

```
#[test]
fn runtime_apply_allows_retroactive_fork_activation() {
    let chain_id = 9_300_001u64;
    RuntimeUpgradeRegistry::clear_chain(chain_id);

    const HEAD: u64 = 2_000_000;
    const HISTORICAL: u64 = HEAD - 500;

    let config = RollupConfig {
        l2_chain_id: chain_id.into(),
        upgrades: UpgradeConfig {
            base: BaseUpgradeConfig {
                beryl: Some(HEAD - 5_000),
                azul: None,
                ..Default::default()
            },
            ..Default::default()
        },
        ..Default::default()
    };
    assert!(!config.is_base_azul_active(HISTORICAL));
    assert!(config.is_beryl_active(HISTORICAL));
}
```



```

UpgradeSignalRuntimeApplier::apply_schedule(
    chain_id,
    &schedule(&[(BaseUpgrade::Azul, HEAD - 1_000), (BaseUpgrade::Beryl, 0)]),
);

assert!(config.is_base_azul_active(HISTORICAL));
assert!(!config.is_beryl_active(HISTORICAL));

RuntimeUpgradeRegistry::clear_chain(chain_id);
}

```

Run: `cargo test -p base-upgrade-signal --lib runtime_apply_allows_retroactive_fork_activation`.

Recommendation: Add an activation-safety gate in `UpgradeSignalRefresher::apply` (or `apply_schedule_to_sink`) parameterized by the node's current head timestamp: reject activations at or before head unless byte-identical to the value already in effect, and reject clears of upgrades whose current activation is already at or before head. Surface rejections as a typed error and metric. Apply the same rule on startup against the persisted head so a restart cannot adopt a schedule that contradicts on-disk history.

Coinbase: Fixed in [PR-409](#).

Cantina Managed: Verified fix.

3.2.4 Default finalized polling can learn emergency delays only after the superseded activation

Severity: Medium Risk.

Context: [upgrade_signal.rs#L124-L139](#), [upgrade_signal.rs#L159-L185](#), [types.rs#L52-L61](#).

Description: `RuntimeAdmin` defaults to the finalized L1 tag and polls it every 15 minutes (`crates/utilities/upgrade-signal/src/config/types.rs:55`). The source comment models finalized state as a two-epoch lag. At 32 slots per epoch and 12 seconds per slot, a schedule change takes about 12 minutes 48 seconds to finalize, then may wait almost another 15 minutes for the next poll. Worst-case observation latency is therefore about 27 minutes 48 seconds.

The incident responder may delay an upgrade at any point before its current activation. The one-hour notice floor constrains the replacement timestamp, not how much time remains until the old timestamp. A delay submitted ten minutes before the old activation is valid, but it cannot even become finalized before that activation.

Nodes with different poll phases can then disagree: some activate the old fork, others honor the delay. That can lead to invalid block production, rejection of canonical blocks, or a halt, without any malicious privileged action.

Proof of concept: The following test is in `crates/utilities/upgrade-signal/src/config/types.rs`. It shows that the default finalized observation lag exceeds a ten-minute-prior delay window.

```

#[test]
fn finalized_polling_can_miss_emergency_delay() {
    assert_eq!(UpgradeSignalBlockTag::default(), UpgradeSignalBlockTag::Finalized);

    let two_epoch_finality_lag = Duration::from_secs(12) * (2 * 32);
    let poll_lag = UpgradeSignalBlockTag::Finalized.poll_interval();
    let worst_case_observation_lag = two_epoch_finality_lag + poll_lag;

    assert_eq!(worst_case_observation_lag, Duration::from_secs(1_668));

    let old_activation_in = Duration::from_secs(10 * 60);

```



```

assert!(old_activation_in < worst_case_observation_lag);
assert_eq!(worst_case_observation_lag - old_activation_in,
    ↪ Duration::from_secs(1_068));
}

```

Run:

```
cargo test -p base-upgrade-signal --lib finalized_polling_can_miss_emergency_delay
```

Expected result: the test passes. Default finalized lag is 1,668 seconds, so a delay submitted 10 minutes before activation is learned 1,068 seconds after that activation.

Recommendation: Treat schedule observation as a latency budget, not only a reorg-safety choice. Polling finalized every L1 slot removes the extra 15-minute phase delay but not finality itself.

Emergency delays also need either enough on-chain lead time, a safe/latest fast path with explicit reorg handling, or a node-side freshness gate near activation. A node should not cross a known activation boundary when its schedule view is too old to exclude a pending or just-finalized delay.

Coinbase: Fixed in [PR-409](#).

Cantina Managed: Verified fix. The relevant protection is primarily the 30-minute freeze on the old activation, not merely the one-hour notice for the new timestamp.

3.2.5 Runtime schedule updates leave Reth's P2P fork filter stale, partitioning running and restarted nodes

Severity: Medium Risk.

Context: [spec.rs#L350-L352](#), [upgrade_signal.rs#L174](#).

Description: The execution client applies new L1 upgrade schedules while running, but Reth creates its P2P `ForkFilter` only during startup. A running node therefore executes the updated rules while still advertising the old ForkID; a restarted node advertises the updated ForkID.

After activation, these nodes classify each other as incompatible and reject their P2P connection, potentially partitioning execution nodes and disrupting sync or recovery.

Proof of Concept: Add this test to the existing test module in `crates/execution/chainspec/src/spec.rs`:

```

#[test]
fn audit_poc_runtime_scheduled_fork_partitions_cached_and_fresh_filters() {
    let chain_id = 9_100_099;
    RuntimeUpgradeRegistry::clear_chain(chain_id);

    let spec = BaseChainSpecBuilder::default()
        .chain(Chain::from_id(chain_id))
        .genesis(Genesis::default())
        .with_fork(EthereumHardfork::Osaka, ForkCondition::Never)
        .with_fork(BaseUpgrade::Azul, ForkCondition::Never)
        .build();

    // Filter cached by a node before Azul is scheduled.
    let mut running = spec.fork_filter(Head {
        timestamp: 41,
        ..Default::default()
    });

    RuntimeUpgradeRegistry::set_activation_timestamp(
        chain_id,

```



```

        BaseUpgrade::Azul,
        42,
    );

    // Filter created by a node restarted after the update.
    let mut restarted = spec.fork_filter(Head {
        timestamp: 41,
        ..Default::default()
    });

    assert_eq!(running.current().next, 0);
    assert_eq!(restarted.current().next, 42);

    running.set_head(Head {
        timestamp: 43,
        ..Default::default()
    });
    restarted.set_head(Head {
        timestamp: 43,
        ..Default::default()
    });

    assert!(running.validate(restarted.current()).is_err());
    assert!(restarted.validate(running.current()).is_err());

    RuntimeUpgradeRegistry::clear_chain(chain_id);
}

```

Run:

```

cargo test -p base-execution-chainspec \
    audit_poc_runtime_scheduled_fork_partitions_cached_and_fresh_filters

```

The test passes, proving both nodes reject each other after activation.

Recommendation: When applying a runtime schedule, atomically rebuild Reth's P2P fork filter, advertised status, and ENR ForkID, then re-handshake affected peers; if safe live replacement is unsupported, require a coordinated restart before activation.

Coinbase: Fixed in [PR-4663](#) and [PR-5017](#).

Cantina Managed: Verified fix. The initial fix in [PR-4663](#) substantially fixes the issue, but its change detector uses the 32-bit EIP-2124 Fork ID as an exact schedule identifier. Because Fork IDs can collide, two different schedules may compare equal and suppress the required P2P filter refresh. This was identified during the fix review and is later fixed in [PR-5017](#).

3.2.6 EIP-8130 activation can disable ZK challenges before compatible verifier rotation

Severity: Medium Risk.

Context: *(No context files were provided by the reviewer).*

Description: The audited SP1 range guest inherits `base-proof` with default features disabled and does not enable a feature that activates the standard-library EVM path. In this feature graph, `base-common-evm` is compiled without `std`. Its transaction dispatcher detects an EIP-8130 transaction and deliberately returns `EIP-8130 execution is unavailable in no_std builds` instead of executing it.

The range program drives that dispatcher while deriving every L2 block in the requested proof interval. Therefore, once an interval contains a type `0x79` transaction, execution aborts before the guest commits its public values and no proof can be generated from that range-program ELF.



This incompatibility crosses an independently administered onchain boundary. `ProtocolVersions.registerUpgrade` and `setTimestamp` can schedule the upgrade that admits EIP-8130 transactions without checking the proof system. Meanwhile, each `AggregateVerifier` implementation stores `ZK_RANGE_HASH` and `ZK_AGGREGATE_HASH` as constructor immutables, so an ELF rebuilt with EIP-8130 support has a new approved hash and requires verifier rotation. `AggregateVerifier.challenge` accepts only `ProofType.ZK`.

If EIP-8130 activates before a compatible verifier implementation is registered and respected, any challenge range containing an EIP-8130 transaction is unprovable under the accepted program hash. A later rotation cannot repair games created against the incompatible implementation. Consequently, a wrong TEE-backed proposal covering such a range loses its permissionless ZK safety backstop and can resolve unless the Guardian blacklists or retires it.

Operationally registering the compatible implementation before the activation timestamp is the intended mitigation. The corresponding `AggregateVerifier` revision rejects a game whose ending L2 timestamp is still ahead of L1, so a timely rotation can cover every post-activation game. That gate constrains game creation; it does not make `ProtocolVersions` enforce verifier readiness, and the audited range program was itself incompatible. Exploitation requires both a missed or incorrect operational rotation and an invalid TEE-backed proposal, so the issue is rated Medium rather than High.

Proof of concept: The feature configuration in the audited checkout is sufficient to establish the incompatible guest build:

```
crates/proof/succinct/programs/Cargo.toml:40
base-proof = { path = "../../..../proof/proof", default-features = false }

crates/proof/succinct/programs/range/ethereum/Cargo.toml:14
base-proof.workspace = true

crates/proof/proof/Cargo.toml:79
"base-common-evm/std",
```

The last entry is reachable only through `base-proof`'s `std` feature, which the range guest does not enable. When the derived block contains EIP-8130, `BaseEvm::transact_raw` reaches the following compiled branch:

```
if tx.is_eip8130() {
    #[cfg(not(feature = "std"))]
    {
        let _ = tx;
        return Err(EVMError::Transaction(BaseTransactionError::eip8130(
            "EIP-8130 execution is unavailable in no_std builds",
        )));
    }
}
```

The error propagates through `WitnessExecutor::run`; `run_range_program` unwraps that result before `sp1_zkvm::io::commit`, so the guest terminates without producing provable public values.

Onchain, the challenge path cannot fall back to the TEE program or another proof type:

```
ProofType proofType = ProofType(uint8(proofBytes[0]));
if (proofType != ProofType.ZK) revert InvalidProofType();
```

Recommendation: Treat proof-system readiness as a hard prerequisite for transaction-type activation. Build the range guest with the narrow EVM `std` feature, exercise a real EIP-8130 block through the production SP1 proving pipeline, derive the resulting range and aggregation hashes, and register the compatible `AggregateVerifier` implementation before scheduling activation.



Enforce this ordering in the upgrade coordinator or deployment tooling and preferably onchain by binding each scheduled execution upgrade to a verifier implementation or readiness commitment that covers it. Do not allow the activation transaction unless the compatible implementation is already registered and respected. Keep the EIP-8130 SP1 system test as a release gate for every future range-program rotation.

Coinbase: Fixed in [PR-4646](#).

Cantina Managed: Verified fix. The fix enables EIP-8130 execution in SP1, but verifier rotation before activation remains operationally coordinated rather than enforced onchain.

3.3 Low Risk

3.3.1 Standalone CL and EL can run with different upgrade schedules without detection

Severity: Low Risk.

Context: [standard_node.rs#L282-L299](#).

Description: Standalone execution and consensus services independently read and apply the L1 upgrade schedule during startup:

- The EL performs its own startup read and applies it to its chain specification.
- The CL separately reads and applies the schedule to its rollup configuration.

Each read is internally pinned to one L1 block, but there is no requirement that both services use the same block, endpoint, schedule, or application mode. Consequently, they can start with different schedules when:

- The finalized L1 head advances between the two reads.
- The configured RPC endpoints return different or stale L1 states.
- One service uses `StartupApply` while the other uses `MetricsOnly`.
- One service successfully refreshes while the other experiences persistent read failures.

This disagreement is not detected through the Engine API. Furthermore, `StartupApply` only applies once and subsequent polling is metrics-only. Therefore, a mismatch can remain for the lifetime of both processes unless they are restarted or manually reconfigured.

At an affected activation timestamp, the CL may derive or validate batches using one set of fork rules while the EL constructs and validates payloads using another. For upgrades such as `Cobalt` that affect both layers, this can cause payload rejection, incompatible state transitions, failed proofs, or an inability to advance the L2 chain.

Integrated commands reduce the initial-read race by reading once and applying the resulting schedule to both clients. However, standalone deployments and independently configured runtime polling remain unprotected.

Recommendation: Use one pinned schedule artifact for both services.

Coinbase: Documenting as future work. Collapse polling into one actor once there's a unified Base binary.

Cantina Managed: Acknowledged.

3.3.2 Live monitor misses schedule shrinkage following an L1 reorganization

Severity: Low Risk.

Context: [state.rs#L130](#).



Description: The `ProtocolVersions` contract's upgrade registry is append-only under normal operation. However, when the node reads from the latest L1 block, it may temporarily observe a newly registered upgrade whose registration is subsequently removed by an L1 reorganization.

The live monitor processes only signals present in the newly returned schedule. If the schedule becomes shorter while its remaining prefix is unchanged, no signal is classified as changed, and the monitor does not submit the shorter schedule for application.

For example:

```
Observed before reorg: [A, B]
Canonical after reorg: [A]
```

Because A is unchanged and B is absent rather than modified, the monitor reports no update. In runtime-admin mode, B's previously applied activation can consequently remain in the runtime registry until a restart, manual refresh, or another detectable schedule change.

If the stale activation is reached, the affected node may apply protocol rules that are no longer scheduled by the canonical L1 state.

Recommendation: Track and compare each complete successfully read schedule, including its vector length, instead of updating only entries present in the new vector.

Retain the source L1 block number and hash so reorganizations and stale observations can be identified. Keep read failures distinct from successfully read empty schedules to prevent an RPC failure from being interpreted as schedule removal.

When a shorter canonical schedule is observed, either atomically apply it so removed tail entries are cleared or fail closed if schedule shrinkage is considered unsupported. Logging the shrinkage while continuing should be avoided because it preserves the stale activation.

Coinbase: Acknowledged and documented in [PR-4660](#).

Cantina Managed: Acknowledged. The added documentation looks good.

3.3.3 Unknown scheduled upgrades are ignored based on an unenforced compatibility assumption

Severity: Low Risk.

Context: [contract.rs#L145](#), [contract.rs#L291-L301](#).

Description: When the L1 `ProtocolVersions` contract returns more schedule entries than the binary recognizes, the node logs a warning and truncates the unknown tail by zipping the returned timestamps with `BaseUpgrade::CONTRACT_VARIANTS`.

Consequently, an unknown entry is discarded before startup or runtime protocol-version validation can determine whether its activation timestamp is zero or nonzero. A zero timestamp represents an unscheduled upgrade and can safely be ignored. A nonzero timestamp, however, represents an upgrade that is scheduled or already active and whose rules the binary does not implement.

The design assumes that the contract's global `minimumProtocolVersion` will reject incompatible binaries. This assumption is not enforced by the contract: `registerUpgrade()` accepts `minProtocolVersion = 0`, leaving the existing minimum unchanged, while `setTimestamp()` can schedule a registered upgrade without updating the minimum.

If governance schedules an unknown upgrade without an adequate minimum-version update, an old node can continue beyond activation using obsolete consensus rules. Updated and outdated nodes may then disagree on block validity or state transitions, potentially causing consensus divergence.

Recommendation: Inspect all timestamps beyond `BaseUpgrade::CONTRACT_VARIANTS.len()` before truncating or mapping the schedule.

Unknown entries with a zero timestamp may be retained for observability and tolerated. Any unknown entry with a nonzero timestamp should cause startup and runtime schedule application to fail closed.



Alternatively, future unknown entries may be retained with an enforced halt before their activation timestamp, although immediate rejection is simpler and safer.

The node should not rely exclusively on `minimumProtocolVersion` to detect unknown protocol capabilities. As defense in depth, the contract could also require a compatible minimum-version update whenever a newly registered upgrade is scheduled.

Coinbase: Failing closed on unrecognized future upgrades would risk halting nodes over a governance-side mistake; we'd rather stay available.

Cantina Managed: Acknowledged.

3.3.4 Upgrade schedule reads are not bound to the configured L1 chain

Severity: Low Risk.

Context: [contract.rs#L82](#).

Description: The upgrade-signal reader obtains a block hash from the operator-supplied L1 RPC and pins its contract calls to that hash. This guarantees that `getSchedule()` and `minimumProtocolVersion()` are read from the same RPC-reported state, but it does not verify that the state belongs to the intended L1 chain.

The reader does not compare `eth_chainId` or the supplied contract address against the `l1_chain_id` and `protocol_versions_address` already present in the rollup configuration. Consequently, an incorrectly configured or compromised endpoint can return a schedule that is accepted and, in startup-apply or runtime-admin mode, used to modify the node's activation rules.

Recommendation: Before applying a schedule, verify that the RPC's chain ID matches the configured L1 chain ID and that the contract address matches the expected `protocol_versions_address`. The contract should also expose a schema identifier that the node verifies before decoding positional schedule entries.

Where the RPC is not fully trusted, obtain the schedule from a locally verified L1 view, verify contract state proofs against an authenticated L1 header, or require agreement from multiple independent endpoints.

Coinbase: Not unique to upgrade-signal; no L1 RPC consumer validates chain ID today, so fixing it here in isolation wouldn't address the broader pre-existing risk.

Cantina Managed: Acknowledged.

3.3.5 Upgrade-signal reads lack consistent resource limits

Severity: Low Risk.

Context: [upgrade_signal.rs#L35](#), [contract.rs#L115](#).

Description: The upgrade-signal reader does not independently enforce a request deadline, response-size limit, or maximum schedule length. Execution paths construct a default HTTP provider without a total request timeout, while consensus paths rely on a shared 15-second transport timeout.

Additionally, `getSchedule()` returns a dynamic ABI array that is fully received and decoded into a `Vec<u64>` before its length is inspected or unknown entries are ignored. A stalled or compromised operator-configured RPC could therefore hang an execution-side schedule read or return an oversized response that consumes excessive memory and potentially disrupts the node.

Recommendation: Use a hardened provider configuration for every upgrade-signal path with explicit connection, read, and total request deadlines. Enforce a response-byte limit at the transport boundary and validate the ABI array's declared length against a conservative maximum before allocating or decoding its elements.

Schedule reads should also support cancellation during shutdown. Retried requests should remain bounded and use jittered backoff to prevent synchronized retry traffic across multiple nodes.



Coinbase: Fixed in [PR-4421](#).

Cantina Managed: Verified fix.

3.3.6 The `UncommittedZenithUpgrade` rejection uses the pin timestamp rather than the claim timestamp

Severity: Low Risk.

Context: [proof_manager.rs#L299-L319](#), [boot.rs#L398-L405](#).

Description: Boot rejects Zenith when `zenith_time <= l2_schedule_timestamp` ([crates/proof/proof/src/boot.rs:398](#)). That timestamp comes from `schedule_l2_block_number`, which is only required to be greater than or equal to the claimed block.

Challenger and fork-dispute paths pin intermediate subranges to the game's final L2 block so every range commits the same `scheduleId`. A pre-Zenith claim can then fail with `UncommittedZenithUpgrade` only because the game endpoint is post-Zenith, even though the proven range never reaches Zenith. The same range loads if the pin equals the claim block.

Mainnet and Sepolia do not schedule Zenith today. Zenith is the experimental slot and can be set in genesis on custom or development chains. On those chains the challenger pin horizon bricks counter-proofs for every pre-Zenith subrange.

Proof of concept: The following test is in [crates/proof/proof/src/boot.rs](#). Claim block 150 is before Zenith; pinning to game block 200 rejects, pinning to the claim block accepts.

```
#[tokio::test]
async fn zenith_rejection_uses_pin_timestamp_not_claim() {
    // Claim 150 => t=300. Zenith at 350 is still in the future for this range.
    // Pin to game block 200 => t=400, which makes Zenith look active.
    let err = BootInfo::load(&oracle).await.expect_err("pre-Zenith claim rejected
    ↳ solely due to a later pin");
    assert!(matches!(err, OracleProviderError::UncommittedZenithUpgrade));

    let boot = BootInfo::load(&claim_pin).await.expect("claim-equal pin should
    ↳ accept the range");
    assert_eq!(boot.rollup_config.upgrades.base.zenith, None);
}
```

Run:

```
cargo test -p base-proof --lib zenith_rejection_uses_pin_timestamp_not_claim
```

Recommendation: Evaluate Zenith against the claimed range timestamp, not the schedule pin timestamp. Keep pinning the game-wide `scheduleId` to the ending block, but do not let that later horizon reject an unsupported experimental fork that the proven range never reaches.

Coinbase: Fixed in [PR-4531](#).

Cantina Managed: Verified fix.

3.3.7 Empty `getSchedule` never auto-applies and differs from all-zero Never / `clear_chain` encodings

Severity: Low Risk.

Context: [upgrade_signal.rs#L87-L107](#), [sink.rs#L41-L53](#), [state.rs#L130-L151](#).



Description: There are three encodings of "no activation", and they do not mean the same thing.

A successful empty `getSchedule()` maps to zero signals. The live monitor then emits no `Initialized` or `Changed` update, so `RuntimeAdmin` never auto-applies.

A full-length all-zero schedule installs explicit `UpgradeActivation::Never` overrides, which suppress base activations.

A manual apply of an empty schedule hits `updates.is_empty()` in `RuntimeRegistrySink::finalize` and calls `clear_chain`. That removes overrides and restores base or genesis activations, which is the opposite of the all-zero `Never` encoding.

Production `ProtocolVersions` is append-only after registration, so an empty array after a healthy schedule is not the usual mainnet path. The practical issue is administrative: `admin_refreshUpgradeSignal` bypasses the monitor and applies directly. Against an uninitialized proxy, a freshly deployed contract, or a mock whose schedule was never set, one admin refresh discards every runtime override and returns success. Live auto-apply and manual refresh therefore disagree on what empty means.

Proof of concept: The following test is in `crates/utilities/upgrade-signal/src/runtime/sink.rs`.

```
#[test]
fn empty_schedule_auto_apply_encoding_skew() {
    let chain_id = 9_000_043u64;
    RuntimeUpgradeRegistry::clear_chain(chain_id);

    let healthy = AlloyUpgradeSignalReader::map_schedule(
        &[10, 20, 30, 40, 50, 60, 70, 0, 80, 90, 100, 110, 120],
        U256::from(7),
        1,
    );
    UpgradeSignalRuntimeApplier::apply_schedule(chain_id, &healthy);
    assert_eq!(
        RuntimeUpgradeRegistry::activation(chain_id, BaseUpgrade::Cobalt),
        Some(UpgradeActivation::Timestamp(120))
    );

    let empty = AlloyUpgradeSignalReader::map_schedule(&[], U256::from(7), 1);
    assert!(empty.signals.is_empty());
    UpgradeSignalRuntimeApplier::apply_schedule(chain_id, &empty);
    assert_eq!(RuntimeUpgradeRegistry::overrides(chain_id), None);

    let zeros = AlloyUpgradeSignalReader::map_schedule(&[0; 13], U256::from(7), 1);
    UpgradeSignalRuntimeApplier::apply_schedule(chain_id, &zeros);
    assert_eq!(
        RuntimeUpgradeRegistry::activation(chain_id, BaseUpgrade::Cobalt),
        Some(UpgradeActivation::Never)
    );

    RuntimeUpgradeRegistry::clear_chain(chain_id);
}
```

Run:

```
cargo test -p base-upgrade-signal --lib empty_schedule_auto_apply_encoding_skew
```

The passing test shows empty apply clears overrides, while `[0; 13]` installs `Never`.

Recommendation: Reject a successful empty `getSchedule` with a typed error instead of mapping it to an empty schedule. Do not infer a full clear from `updates.is_empty()`. If a full clear is required,



make it explicit or install `Never` for every contract-backed upgrade. Keep read failures distinct from empty success.

Coinbase: Fixed in [PR-4526](#).

Cantina Managed: Verified fix.

3.3.8 Optional boot preimage reads silently default on all oracle errors

Severity: Low Risk.

Context: [boot.rs#L302](#).

Description: `BootInfo::load` treats every error from four optional preimage reads as if the key were absent and substitutes zero or another fallback value. This also swallows operational failures such as timeouts, I/O errors, and a closed oracle instead of aborting the load. The proof may therefore continue with unintended inputs, waste substantial work, and ultimately produce a rejected result rather than exposing the real failure.

Recommendation: Default only on the explicit key-not-found error and propagate every other oracle error.

Coinbase: Fixed in [PR-4521](#).

Cantina Managed: Verified fix.

3.3.9 `AggregateVerifier` permanently pins a still-mutable future schedule

Severity: Low Risk.

Context: [rollup.rs#L334-L340](#), [boot.rs#L351-L407](#).

Description: `AggregateVerifier.initializeWithInitData` derives a timestamp from the claimed L2 block, stores `PROTOCOL_VERSIONS.activatedScheduleId(claimTimestamp)`, and never updates that value. It does not require the claim timestamp to be at or before the current L1 `block.timestamp`.

This crosses two clock domains unsafely. Post-Fjord batch validation permits L2 timestamps up to 1,800 seconds ahead of their L1 origin, while `ProtocolVersions.setTimestamp` and `delayTimestamp` permit an activation at `T` to be cleared or delayed until L1 time reaches `T`. A sequencer can therefore publish and prove a claim with L2 timestamp `>= T` while L1 time is still `< T`, create or front-run an emergency transaction with a game that pins the old schedule, and then let the clear or delay execute.

After the schedule changes, proofs produced from the canonical schedule commit the new `scheduleId`, but every verification path in the existing game continues hashing its stored old `scheduleId`. A canonical ZK proof cannot challenge or supplement that game, while its pre-change TEE proof can still resolve and the rolled-back output can become the anchor unless the Guardian manually blacklists or retires it.

Proof of Concept: Add the following test as `test/L1/proofs/AggregateVerifierScheduleRollback.t.sol` in the matching `base-contracts` checkout:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.15;

import { BaseTest } from "test/L1/proofs/BaseTest.t.sol";

import { IVerifier } from "interfaces/L1/proofs/IVerifier.sol";
import { AggregateVerifier } from "src/L1/proofs/AggregateVerifier.sol";
import { Claim } from "src/libraries/bridge/LibUDT.sol";
import { Hash } from "src/libraries/bridge/Types.sol";
```



```
contract AggregateVerifierScheduleRollbackTest is BaseTest {
    uint64 private constant POST_FJORD_MAX_SEQUENCER_DRIFT = 1800;

    function test_futureL2GameSurvivesCanonicalScheduleClear_succeeds() public {
        uint64 activationTimestamp = uint64(BLOCK_INTERVAL * L2_BLOCK_TIME);
        uint64 l2Lead = activationTimestamp - uint64(block.timestamp);
        assertLt(block.timestamp, activationTimestamp);
        assertLe(l2Lead, POST_FJORD_MAX_SEQUENCER_DRIFT);

        protocolVersions.registerUpgrade(activationTimestamp, 1);
        bytes32 pinnedScheduleId =
            ↪ protocolVersions.activatedScheduleId(activationTimestamp);
        assertNotEq(pinnedScheduleId, bytes32(0));

        currentL2BlockNumber += BLOCK_INTERVAL;
        Claim rootClaim = Claim.wrap(keccak256(abi.encode(currentL2BlockNumber)));
        AggregateVerifier game = _createAggregateVerifierGame(
            TEE_PROVER,
            rootClaim,
            currentL2BlockNumber,
            address(anchorStateRegistry),
            _generateProof("pre-clear", AggregateVerifier.ProofType.TEE)
        );
        assertEq(game.scheduleId(), pinnedScheduleId);

        protocolVersions.setTimestamp(0, 0);
        bytes32 canonicalScheduleId =
            ↪ protocolVersions.activatedScheduleId(activationTimestamp);
        assertEq(canonicalScheduleId, bytes32(0));
        assertEq(game.scheduleId(), pinnedScheduleId);

        bytes memory zkProof = bytes("canonical-proof");
        bytes32 canonicalJournal = _zkJournal(game, canonicalScheduleId);
        bytes32 staleGameJournal = _zkJournal(game, pinnedScheduleId);
        assertNotEq(canonicalJournal, staleGameJournal);

        vm.mockCall(
            address(zkVerifier),
            abi.encodeCall(IVerifier.verify, (zkProof, ZK_AGGREGATE_HASH,
            ↪ canonicalJournal)),
            abi.encode(true)
        );
        vm.mockCall(
            address(zkVerifier),
            abi.encodeCall(IVerifier.verify, (zkProof, ZK_AGGREGATE_HASH,
            ↪ staleGameJournal)),
            abi.encode(false)
        );
        assertTrue(zkVerifier.verify(zkProof, ZK_AGGREGATE_HASH, canonicalJournal));

        vm.expectRevert(AggregateVerifier.InvalidProof.selector);
        vm.prank(ZK_PROVER);
        game.verifyProposalProof(abi.encodePacked(uint8(AggregateVerifier.ProofType |
            ↪ .ZK),
            ↪ zkProof));

        vm.warp(block.timestamp + game.SLOW_FINALIZATION_DELAY());
        game.resolve();
        vm.warp(block.timestamp + 1);
        game.closeGame();

        (Hash anchorRoot, uint256 anchorBlockNumber) =
            ↪ anchorStateRegistry.getAnchorRoot();
        assertEq(anchorRoot.raw(), rootClaim.raw());
    }
}
```



```

    assertEq(anchorBlockNumber, currentL2BlockNumber);
}

function _zkJournal(AggregateVerifier game, bytes32 scheduleId) private view
↳ returns (bytes32) {
    bytes memory proposal = abi.encodePacked(
        ZK_PROVER,
        game.l1Head().raw(),
        game.startingRootHash().raw(),
        uint64(game.startingBlockNumber()),
        game.rootClaim().raw(),
        uint64(game.l2SequenceNumber())
    );
    bytes memory proofConfig =
        abi.encodePacked(game.intermediateOutputRoots(), CONFIG_HASH,
        ↳ ZK_RANGE_HASH, scheduleId);
    return keccak256(bytes.concat(proposal, proofConfig));
}
}

```

Run:

```

forge test \
  --match-path test/L1/proofs/AggregateVerifierScheduleRollback.t.sol \
  --match-test test_futureL2GameSurvivesCanonicalScheduleClear_succeeds \
  -vv

```

```

[PASS] test_futureL2GameSurvivesCanonicalScheduleClear_succeeds()
Suite result: ok. 1 passed; 0 failed; 0 skipped

```

The test proves that the L2 claim is within the protocol's drift window, the game retains the pre-clear schedule, a proof valid for the canonical post-clear journal is rejected through that game, and the stale game nevertheless resolves and becomes the anchor.

Recommendation: Do not create a game for a schedule that L1 can still mutate: require the derived L2 claim timestamp to be no greater than the game-creation L1 timestamp, or freeze schedule changes early enough to cover the maximum L2 lead and proof-submission window. Additionally, bind each game to an immutable schedule snapshot and automatically invalidate games whose pinned schedule is superseded by an authorized clear or delay; do not rely solely on manual Guardian blacklisting.

Coinbase: Fixed in [PR-406](#).

Cantina Managed: Verified fix.

3.4 Informational

3.4.1 Contract-valid dependency holes cause inconsistent CL and EL fork activation

Severity: Informational.

Context: [rollup.rs#L254-L277](#), [spec.rs#L440-L460](#).

Description: The L1 `ProtocolVersions` contract can contain a schedule where a later upgrade is scheduled while one of its required predecessors is disabled.

Specifically, the `ProxyAdmin` owner may clear a future activation using `setTimestamp(id, 0)`. The ordering and successor checks execute only when the new timestamp is non-zero, so clearing `Canyon` and `Delta` while leaving `Ecotone` scheduled is accepted by `ProtocolVersions.setTimestamp`.

The resulting schedule is interpreted differently:



- The CL cascades activation from later upgrades into their predecessors. Thus, `Ecotone` being active implies `Delta` and `Canyon` are active.
- The EL installs each upgrade's activation condition independently. Thus, `Canyon` remains `Never` even when `Ecotone` is active.

For example:

```
Canyon = Never
Delta   = Never
Ecotone = Timestamp(T)
```

At T:

```
CL: Canyon active = true
EL: Canyon active = false
```

Consequently, the CL may derive payloads assuming earlier fork rules are active while the EL evaluates Canyon-specific execution and validation gates as inactive. This may result in payload rejection, inconsistent state transitions, failed proofs, or the node being unable to follow the canonical chain.

The `scheduleId` does not prevent this because it only commits to the raw timestamps. Both components receive the same committed schedule but assign it different meanings.

Only the `ProxyAdmin` owner can create this condition. This privileged prerequisite reduces likelihood, but an otherwise authorized cancellation or configuration mistake can still violate a critical CL/EL consistency invariant.

Proof of Concept: Add the following test at the end of the existing `#[cfg(test)]` mod tests in `crates/execution/chainspec/src/spec.rs`:

Update the existing test import:

```
use base_common_genesis::{BaseUpgrade, RollupConfig, RuntimeUpgradeRegistry};
```

Then add:

```
#[test]
fn dependency_hole_splits_cl_and_el_activation() {
    const REGOLITH: u64 = 1;
    const ECOTONE: u64 = 100;

    // CL interpretation of the onchain schedule.
    let mut cl = RollupConfig::default();
    cl.clear_upgrade_activation_timestamps();
    cl.set_upgrade_activation_timestamp(BaseUpgrade::Regolith, REGOLITH);
    cl.set_upgrade_activation_timestamp(BaseUpgrade::Ecotone, ECOTONE);

    // EL interpretation of the same schedule.
    let mut el = BaseChainSpec::devnet();
    el.clear_hardfork_activation_timestamps();
    el.try_set_hardfork_activation_timestamp(BaseUpgrade::Regolith,
        ↪ REGOLITH).unwrap();
    el.try_set_hardfork_activation_timestamp(BaseUpgrade::Ecotone,
        ↪ ECOTONE).unwrap();

    assert!(cl.is_canyon_active(ECOTONE));
    assert!(cl.is_delta_active(ECOTONE));

    assert!(!el.is_canyon_active_at_timestamp(ECOTONE));
    assert!(el.is_ecotone_active_at_timestamp(ECOTONE));
}
```



```
}
```

Run:

```
cargo test -p base-execution-chainspec \
  --lib dependency_hole_splits_cl_and_el_activation \
  -- --nocapture
```

The passing assertions establish the issue: the same schedule makes the CL report `Canyon` active while the EL reports `Canyon` inactive.

Recommendation: Introduce one canonical schedule validator shared by EL, CL, and proof code.

Coinbase: Acknowledged.

Cantina Managed: Acknowledged.

3.4.2 Development builds bypass protocol-version compatibility checks

Severity: Informational.

Context: [mod.rs#L49-L51](#), [schedule.rs#L47-L49](#).

Description: The workspace uses version `0.0.0` for ordinary source, development, and CI builds. The upgrade-signal implementation converts this version into `U256::MAX`, intending to prevent development builds from being rejected.

Compatibility is then determined by checking whether the required protocol version is less than or equal to the node's advertised version. Because every `U256` value is less than or equal to `U256::MAX`, a `0.0.0` binary passes every nonzero protocol-version requirement, regardless of which protocol rules it actually implements.

Consequently, a source-built, CI, release-candidate, or incorrectly versioned binary running in startup-apply or runtime-admin mode may accept an upgrade it does not support. It could continue operating until activation, at which point it may lose synchronization or diverge from nodes applying the new protocol rules.

The likelihood is reduced because official releases receive a nonzero Cargo version and upgrade-signal application is opt-in. Nevertheless, the potential impact includes a chain split, making it a good-to-fix issue.

Recommendation: Define an explicit, audited protocol-capability version based on the protocol features implemented by the binary, independently of its Cargo package version.

Treat `0.0.0` and other unset capability values as unsupported and fail closed. Applying modes should also reject schedules containing unknown future upgrades instead of ignoring them.

If development environments require a bypass, place it behind a conspicuous unsafe option restricted to development builds or development chains, with prominent warnings and metrics.

Coinbase: Acknowledged. This is by design. Dev/CI builds carry the latest feature set, so treating 0.0.0 as max-version is correct. Only versioned releases run in production; no real-world impact.

Cantina Managed: Acknowledged.

3.4.3 Schedule commitment does not bind numeric positions to semantic upgrade identities

Severity: Informational.

Context: *(No context files were provided by the reviewer).*



Description: The `ProtocolVersions` contract represents upgrades using sequential numeric IDs and returns their activation timestamps as a positionally ordered array. Human-readable upgrade identities are intentionally maintained offchain.

The node interprets each array position using the locally defined `crates/common/genesis/src/chain/upgrade.rs:133`. Its ordering is explicitly documented as load-bearing because changing it would associate contract timestamps with different hardforks.

Although the contract exposes `scheduleId`, that value commits only to each numeric index and its timestamp:

```
hash(previous_hash, numeric_index, activation_timestamp)
```

It does not commit to the semantic identity assigned to that index, such as `2 = Delta`. Consequently, a client that accidentally reorders two variants could still reproduce the contract's schedule commitment while applying the timestamps to the wrong upgrades.

The runtime reader also obtains only `crates/utilities/upgrade-signal/src/contract.rs:23`. It cannot verify that its local positional mapping matches the semantic mapping intended by the deployed contract.

The current ordering appears correct, and registration is append-only and owner-controlled. Exploitation therefore requires a faulty client release or incorrectly initialized deployment rather than an untrusted user action. This is an architectural hardening issue rather than a presently exploitable vulnerability.

Recommendation: Bind every numeric position to an immutable semantic upgrade identifier. For example, the contract could store a `bytes32` identifier such as `keccak256("delta")` for each registered upgrade or expose a hash of the ordered semantic-ID list.

At startup and during runtime refresh, clients should verify that the contract's known semantic-ID prefix matches their local upgrade mapping before interpreting any timestamps. A mismatch should fail closed.

Additionally, maintain a deployment-backed golden test that compares the client's expected (numeric ID, semantic ID) mapping against an independently maintained canonical manifest or deployed contract. The expected values should not be generated from `CONTRACT_VARIANTS`, since deriving both sides from the same constant would not detect accidental reordering.

Coinbase: Acknowledged. Nice to have, no immediate action.

Cantina Managed: Acknowledged.

3.4.4 Runtime-admin can activate `Bery1` without the required activation admin

Severity: Informational.

Context: [spec.rs#L238](#), [sink.rs#L25](#).

Description: `Bery1` introduces activation-registry precompiles that require a nonzero, chain-specific activation-admin address. The execution chain specification enforces this invariant during construction and startup schedule application: any scheduled `Bery1` activation is rejected when the admin is missing or zero.

The runtime-admin path does not preserve this invariant. `UpgradeSignalRefresher::apply()` performs protocol-version validation and then applies the schedule through `RuntimeRegistrySink`. This generic sink is infallible and accepts every contract-backed upgrade except Zenith; it has only the L2 chain ID and cannot inspect the chain's activation-admin configuration.

The following sequence is therefore possible:

1. A custom chain starts with `Bery1` unscheduled and no activation admin.
2. Startup validation succeeds because `Bery1` is inactive.
3. The node runs with runtime-admin mode enabled.



4. Governance later assigns `Beryl` a nonzero activation timestamp.
5. Automatic polling or `admin_refreshUpgradeSignal` installs `Beryl` in `RuntimeUpgradeRegistry` without validating the admin requirement.
6. Runtime-aware fork checks begin treating `Beryl` as active.

The EVM configuration was constructed without an activation admin. Consequently, `Beryl`'s administrative precompile operations revert rather than granting unauthorized access. More importantly, the node has entered a configuration that startup validation considers invalid.

Runtime overrides are also materialized when the node serves its rollup configuration. At or after `Beryl` activation, proof boot rejects that configuration with `MissingActivationAdminAddress` in `crates/proof/proof/src/boot.rs:409`, potentially preventing proof generation and disrupting settlement or dispute-system liveness.

Proof of Concept: Add the following test before the final closing brace of the existing `#[cfg(test)]` mod tests in `crates/execution/cli/src/upgrade_signal.rs`:

```
#[test]
fn runtime_admin_accepts_beryl_without_activation_admin() {
    let chain_id = 9_100_005;
    RuntimeUpgradeRegistry::clear_chain(chain_id);

    // A custom chain with no configured activation admin and no initial
    // Beryl activation is accepted.
    let mut chain_spec = BaseChainSpec::from(ChainSpec {
        chain: chain_id.into(),
        ..ChainSpec::default()
    });
    let beryl_schedule = versioned_schedule(BaseUpgrade::Beryl, 42);

    // Applying the schedule through the startup chain-spec path correctly
    // rejects Beryl because the activation admin is missing.
    let startup_error =
        ExecutionUpgradeSignal::apply_schedule_to_chain_spec(
            &mut chain_spec,
            &beryl_schedule,
        )
        .unwrap_err();

    assert!(
        startup_error
            .to_string()
            .contains("missing activation admin address")
    );
    assert_eq!(
        chain_spec.fork(BaseUpgrade::Beryl),
        ForkCondition::Never
    );

    // The runtime-admin path applies the identical schedule to the global
    // registry without checking the chain-spec invariant.
    let summary = runtime_refresher(chain_id)
        .apply(&beryl_schedule)
        .unwrap();

    assert_eq!(summary.applied_upgrades, 1);

    // The same chain spec now observes Beryl as active through the runtime
    // registry despite still having no activation admin.
    assert_eq!(
        chain_spec.fork(BaseUpgrade::Beryl),
        ForkCondition::Timestamp(42)
    );
}
```



```

    );
    assert_eq!(
        RuntimeUpgradeRegistry::activation(
            chain_id,
            BaseUpgrade::Beryl
        ),
        Some(UpgradeActivation::Timestamp(42))
    );

    RuntimeUpgradeRegistry::clear_chain(chain_id);
}

```

Run the test from the repository root:

```

cargo test -p base-execution-cli \
    runtime_admin_accepts_beryl_without_activation_admin \
    -- --nocapture

```

Expected output:

```

running 1 test
test upgrade_signal::tests::runtime_admin_accepts_beryl_without_activation_admin ...
↳ ok

test result: ok. 1 passed; 0 failed

```

The passing test demonstrates that the same Beryl schedule is rejected by the startup path but accepted by the runtime-admin path.

Recommendation: Validate the fully materialized candidate schedule against the node's chain-specific capabilities before mutating `RuntimeUpgradeRegistry`.

Coinbase: No live Base chain affected; new chains enable Beryl at genesis with the zero-admin check enforced there. We'll add a similar gate if a future upgrade introduces Beryl-like admin requirements.

Cantina Managed: Acknowledged.

3.4.5 The `minimumProtocolVersion` gate compares packed versions as raw `U256`, inverting prerelease ordering

Severity: Informational.

Context: [mod.rs#L40-L45](#), [schedule.rs#L46-L49](#).

Description: Compatibility is a raw integer comparison of the packed protocol version:

```

signal.protocol_version <= self.node_protocol_version

```

The node packs only `major.minor.patch` and leaves the prerelease field at zero (`crates/utilities/upgrade-signal/src/config/mod.rs:43`). The OP-Stack layout also carries build and prerelease fields. Build should be ignored when ordering, and a prerelease should sort below its release.

A raw `U256` comparison gets the prerelease order backwards. `1.2.3-rc.1` packs with `pre-release = 1`, so it compares greater than final `1.2.3` (`pre-release = 0`). A node running the final release is then rejected by a minimum of the release candidate, and startup apply or runtime refresh can fail for a binary that should be considered sufficient.

Canonical minima currently use a zero prerelease, so this is not the usual production path. It still means the gate does not implement the version order it is documented to follow.



Proof of concept: The following test is in `crates/utilities/upgrade-signal/src/config/schedule.rs`.

```
#[test]
fn protocol_version_gate_compares_raw_u256_inverts_prerelease() {
    let mut config = supported_config();
    config.node_protocol_version =
        ↪ UpgradeSignalDefaults::packed_protocol_version(1, 2, 3);

    let prerelease = packed_protocol_fields(0, 0, 1, 2, 3, 1);
    assert!(matches!(
        config.validate_signal_protocol_version(&signal(prerelease)).unwrap_err(),
        ↪ crate::UpgradeSignalError::UnsupportedProtocolVersion { .. }
    ));
}
```

Run:

```
cargo test -p base-upgrade-signal --lib
↪ protocol_version_gate_compares_raw_u256_inverts_prerelease
```

Recommendation: Compare only the ordered semver fields, with prerelease sorting below the matching release, and ignore build and reserved bits. Do not use raw `U256` ordering for protocol compatibility.

Coinbase: Fixed in [PR-4523](#).

Cantina Managed: Verified fix.

3.4.6 `ScheduleId::pin` doc cites `scheduleId()` while implementing activated-prefix semantics

Severity: Informational.

Context: [schedule_id.rs#L9-L50](#).

Description: Comments say the hash chain mirrors on-chain `ProtocolVersions.scheduleId()` and uses `0` for unscheduled entries. The implementation actually hashes the activated prefix at `12_timestamp`: it finds the last upgrade with `activation <= 12_timestamp`, clears entries above that prefix, then hashes `0..pinned_count`.

That matches `activatedScheduleId`-style semantics, not a full-array `scheduleId()` over every registered upgrade. The algorithm and golden test are consistent with prefix pinning. The wrong Solidity name in the docs is a maintenance hazard if someone later “fixes” Rust to hash the full ladder, including post-prefix zeros.

Existing tests `pin_clears_only_entries_above_activated_prefix` and `schedule_id_is_zero_when_nothing_is_active` already show that entries above the activated prefix are cleared and not hashed.

Recommendation: Point the docs at the exact Solidity entrypoint the verifier uses, and state that only the activated prefix is hashed.

Coinbase: Fixed in [PR-4528](#).

Cantina Managed: Verified fix.

3.4.7 `Clone` does not guarantee transactional sink isolation

Severity: Informational.

Context: [sink.rs#L61-L104](#).



Description: `apply_schedule_to_sink` claims atomic application because it stages updates on `sink.clone()`, but Rust's `Clone` trait does not require an independent copy.

A future sink backed by `Arc<Mutex<_>>`, `Rc<RefCell<_>>`, or external side effects could mutate shared state before a later activation or `finalize` returns an error. The current production sinks are value-owned and safe, but the public generic API does not enforce the transactional guarantee it documents.

Recommendation: Replace the `Clone` assumption with an explicit transactional staging-and-commit abstraction, or seal the trait to audited value-owned sinks.

Coinbase: Valid. Tracking as future hardening.

Cantina Managed: Acknowledged.

3.4.8 Runtime-added upgrades are omitted from fork enumeration

Severity: Informational.

Context: [spec.rs#L332](#).

Description: `forks_iter()` only walks forks present in the startup chain specification. An upgrade added entirely at runtime can affect execution while remaining absent from fork enumeration and downstream `eth_config` responses, misleading RPC clients and monitoring tools.

Recommendation: Enumerate `runtime_hardforks()` instead, and test an initially absent fork becoming scheduled at runtime.

Coinbase: Fixed in [PR-4569](#).

Cantina Managed: Verified fix.